

Modulo 2

Angelina Moi

2026-04-01

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

```
##Primeros comandos de R
```

R es un lenguaje muy similar a MatLab y permite el trabajo con matrices.

```
A=38
A=40
B=60
C=B-A
C
```

```
## [1] 20
```

Uso de datasets o base de datos internos de R

Usando el comando data en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan dados en tablas de las cuales podemos elegir columnas especificas si escribimos \$, luego indicando la informacion especifica que deseamos.

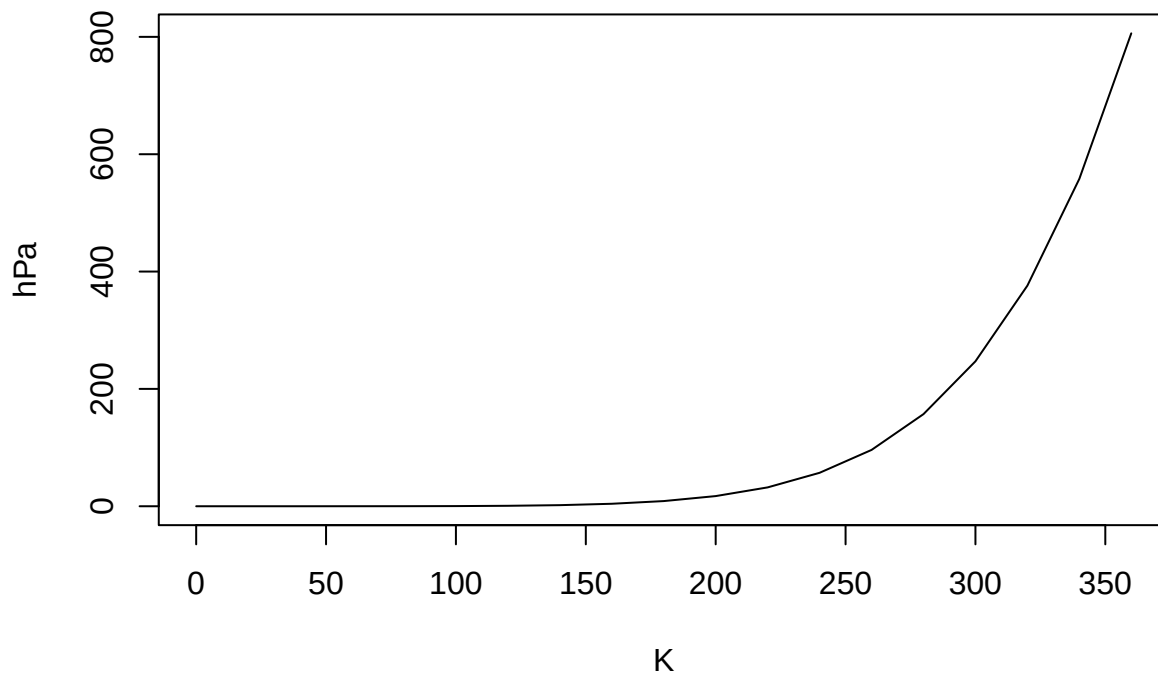
```
summary(cars)
```

```
##      speed          dist
##  Min.   : 4.0      Min.   : 2.00
## 1st Qu.:12.0      1st Qu.: 26.00
##  Median :15.0      Median : 36.00
##   Mean  :15.4      Mean   : 42.98
## 3rd Qu.:19.0      3rd Qu.: 56.00
##   Max.  :25.0      Max.    :120.00
```

Comando Plot

Es un comando que permite graficar cualquier fuente de datos que le asignemos, teniendo también la posibilidad de asignar nombre a las variables o hacer cambios al grafico. En caso de duda de como utilizar el comando podemos escribir **?plot** en la consola, lo cual abrira una ventana que explicara en detalle el comando.

Presion del gas ideal



Para que en grafico me aparezca la funcion con una linea, o puntos o de cualquier forma que yo quiera, escribo `type=letra` para saber que significa cada letra puedo escribir en la consola `?plot`.

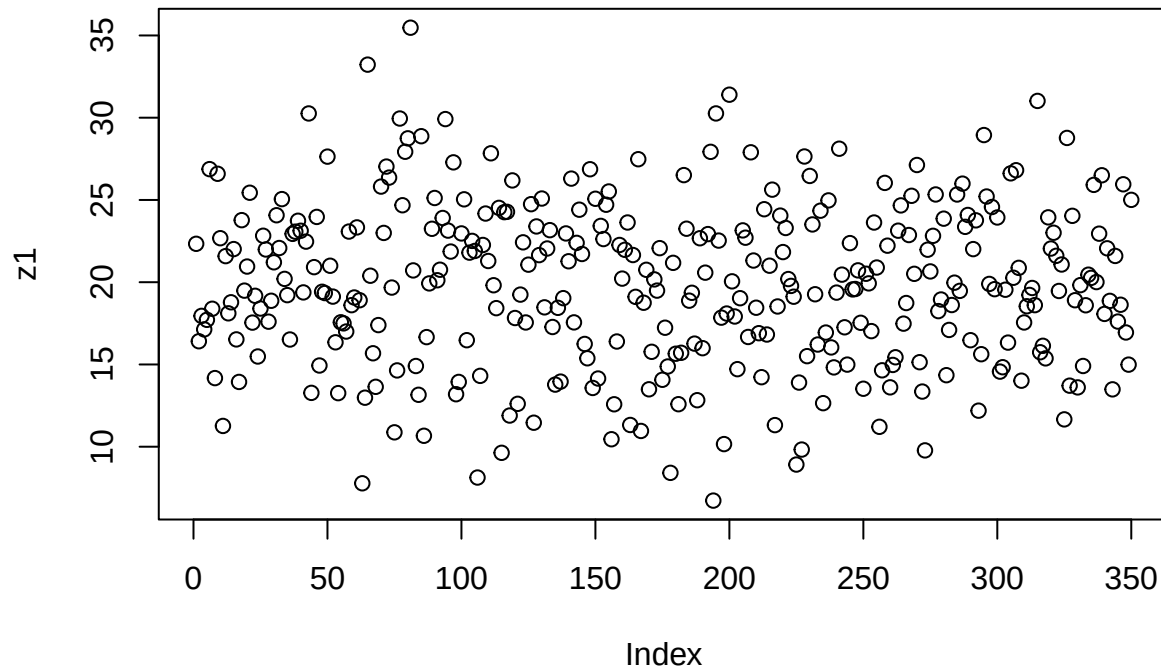
Funciones Estadisticas

```
z1<-rnorm(350,20,5)
z1
```

```
##      [1] 22.335566 16.412052 17.953477 17.131408 17.713317 26.874005 18.386742
##      [8] 14.171828 26.592355 22.675781 11.262666 21.585947 18.108881 18.779580
##     [15] 22.013987 16.523541 13.938210 23.773429 19.488741 20.952886 25.439995
##     [22] 17.545551 19.174150 15.485383 18.400623 22.824864 21.990298 17.602295
##     [29] 18.872081 21.209794 24.071410 22.078755 25.052253 20.201712 19.213956
##     [36] 16.516753 22.940220 23.072920 23.735918 23.127026 19.380456 22.468192
##     [43] 30.257895 13.274002 20.916141 23.969723 14.937068 19.419033 19.340866
##     [50] 27.634391 21.000147 19.126114 16.347494 13.257739 17.565145 17.497436
##     [57] 17.001637 23.079226 18.605367 19.070303 23.331968 18.905601  7.775913
##     [64] 12.980434 33.226089 20.394426 15.680584 13.641307 17.394534 25.813297
##     [71] 22.996144 27.028930 26.359435 19.674469 10.874700 14.638621 29.950889
##     [78] 24.679305 27.930485 28.746029 35.471101 20.717783 14.905701 13.156587
##     [85] 28.875030 10.665408 16.662960 19.927842 23.248242 25.119727 20.119694
##     [92] 20.759596 23.904754 29.915706 23.137846 21.863647 27.285918 13.190699
##     [99] 13.937552 22.965368 25.038129 16.475362 21.797963 22.511611 21.900985
##    [106]  8.127581 14.307597 22.264333 24.174459 21.289816 27.837256 19.815642
##    [113] 18.424120 24.517967  9.630188 24.258851 24.277887 11.895225 26.187335
##    [120] 17.824452 12.607961 19.246664 22.424021 17.552606 21.067420 24.743610
##    [127] 11.459161 23.390273 21.647365 25.087121 18.471551 22.049511 23.164388
##    [134] 17.278157 13.776061 18.442733 13.956974 19.030962 22.965457 21.273620
##    [141] 26.298180 17.557693 22.391521 24.408390 21.715161 16.247204 15.375658
##    [148] 26.867997 13.571070 25.072957 14.145407 23.422987 22.623874 24.709442
```

```
## [155] 25.515547 10.455022 12.580117 16.395903 22.263985 20.219276 21.970204
## [162] 23.630516 11.321309 21.653170 19.110022 27.476966 10.958371 18.752478
## [169] 20.754568 13.490477 15.770563 20.171528 19.490293 22.070430 14.070078
## [176] 17.233731 14.876931 8.412924 21.177847 15.653652 12.584844 15.709382
## [183] 26.511204 23.252798 18.878475 19.357427 16.267421 12.827726 22.671406
## [190] 15.990144 20.581571 22.925355 27.925512 6.725183 30.259513 22.530519
## [197] 17.845868 10.160088 18.096472 31.399288 20.053050 17.915821 14.714739
## [204] 19.027826 23.150011 22.705815 16.670777 27.896992 21.318581 18.448664
## [211] 16.901587 14.227147 24.436936 16.820662 21.003741 25.626983 11.313904
## [218] 18.526373 24.048902 21.832683 23.295914 20.191901 19.776129 19.118924
## [225] 8.915957 13.899393 9.826879 27.642174 15.505064 26.459913 23.512118
## [232] 19.268543 16.212178 24.347581 12.648019 16.940896 24.974645 16.035197
## [239] 14.821969 19.368907 28.114778 20.461810 17.259342 14.991897 22.376704
## [246] 19.562572 19.587477 20.719639 17.532989 13.524912 20.513796 19.929036
## [253] 17.027312 23.628548 20.892817 11.210193 14.644502 26.036881 22.221493
## [260] 13.609671 14.971031 15.437450 23.131728 24.669155 17.481453 18.735261
## [267] 22.868288 25.253439 20.509548 27.125303 15.136888 13.353027 9.776385
## [274] 21.976237 20.655325 22.819308 25.334432 18.243748 18.949278 23.863201
## [281] 14.348728 17.099850 18.613899 19.973111 25.332957 19.473116 25.992754
## [288] 23.366698 24.087415 16.474315 22.013178 23.760848 12.192253 15.624206
## [295] 28.946095 25.209033 19.895564 24.572746 19.577289 23.934408 14.565545
## [302] 14.832515 19.543545 16.328883 26.617028 20.272065 26.807226 20.872297
## [309] 14.013733 17.553357 18.538109 19.225575 19.649729 18.607431 31.020163
## [316] 15.742751 16.134121 15.370805 23.942774 22.046808 23.008462 21.606167
## [323] 19.468204 21.077343 11.661432 28.769854 13.711479 24.036028 18.904638
## [330] 13.605535 19.822352 14.907924 18.598192 20.458628 20.264036 25.917721
## [337] 20.009104 22.950975 26.500352 18.072568 22.061229 18.865804 13.487158
## [344] 21.596840 17.608492 18.630651 25.953583 16.944991 14.990635 25.006690
```

```
plot(z1)
```



```
w1<-length(z1)
w1
```

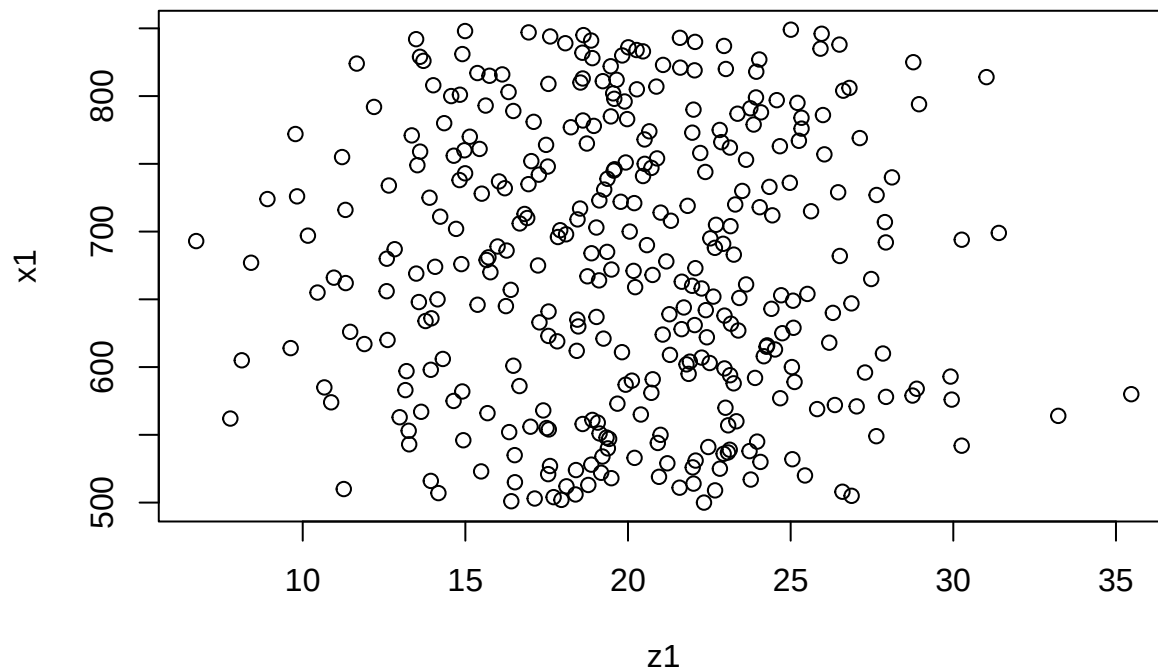
```
## [1] 350
```

```
x1<-(500:849)
```

```
x1
```

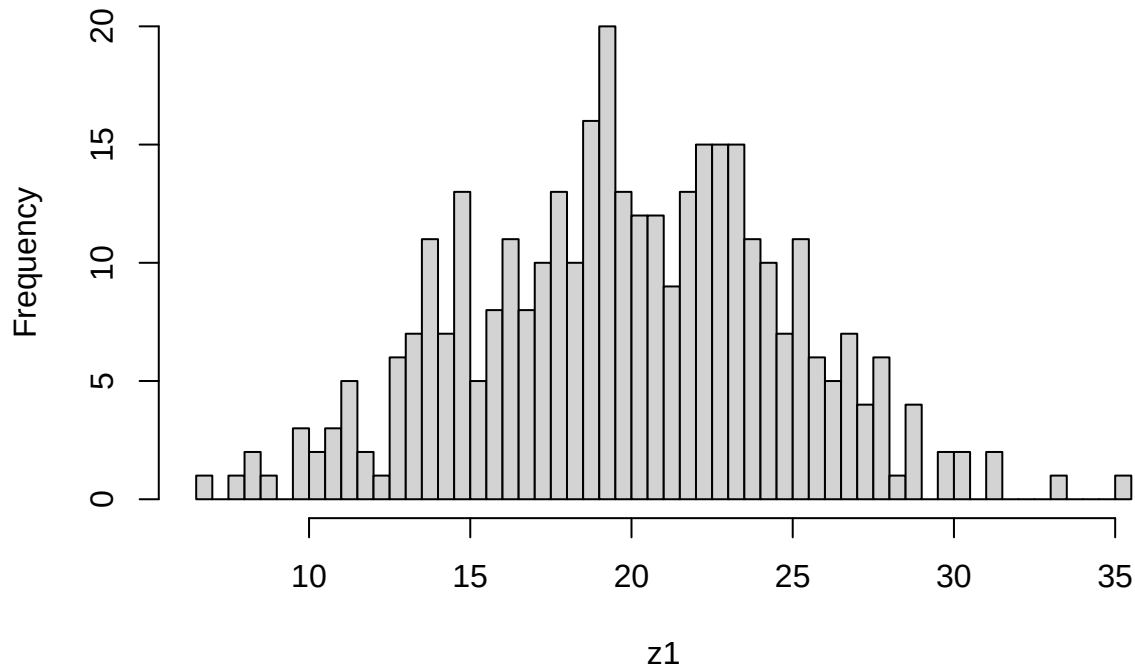
```
## [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
## [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
## [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
## [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
## [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
## [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```
plot(z1,x1)
```



```
hist(z1,main="Histograma de Edades",breaks=50)
```

Histograma de Edades



```
density(z1)
```

```
##
## Call:
## density.default(x = z1)
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.363
##
##      x              y
## Min.   : 2.637   Min.   :9.357e-06
## 1st Qu.:11.868   1st Qu.:1.605e-03
## Median :21.098   Median :1.471e-02
## Mean   :21.098   Mean   :2.703e-02
## 3rd Qu.:30.328   3rd Qu.:5.208e-02
## Max.   :39.559   Max.   :7.692e-02
```

El tamaño de muestra es de 350, valor promedio de 20 y un desvio estandar de 5. Si tengo alguna duda escribo en la consola **?rnorm**.

Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

Ejercicio 1 de algoritmo

Consigna: Las ultimas 3 cifras de mi DNI son 049, crear una variable que tenga ese numero

```
DNI=49
```

Consigna: Crear un vector que tenga todos los numeros enteros desde el 1 al 49

```
Secuencia_dni<-(1:49)
Secuencia_dni
```

```
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
## [26] 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
```

##Ejercicio 2 de algoritmo Calcular la suma de todos los valores del vector Secuencia_dni

```
Total<-0
Valor_final<-length(Secuencia_dni)
for(i in 1:49) {
Total<-Total+i
Total
}
Total
```

```
## [1] 1225
```

##Ejercicio 3 Repetir el ejercicio anterior pero en Python

```
totalp = 0
for i in range(1,50):
    totalp += i
    print(totalp)
```

```
## 1
## 3
## 6
## 10
## 15
## 21
## 28
## 36
## 45
## 55
## 66
## 78
## 91
## 105
## 120
## 136
## 153
## 171
## 190
## 210
## 231
## 253
## 276
## 300
## 325
## 351
## 378
## 406
## 435
## 465
## 496
## 528
## 561
## 595
## 630
```

```
## 666
## 703
## 741
## 780
## 820
## 861
## 903
## 946
## 990
## 1035
## 1081
## 1128
## 1176
## 1225
```

##Ejercicio 4 ¿Cuanto tarda en correr el código del ejercicio 2?

```
inicio<-Sys.time()
Total<-0
Valor_final<-10000000*length(Secuencia_dni)

for (i in 1:Valor_final){
  Total<-Total+i
}
Total
```

```
## [1] 1.2005e+17
```

```
final<-Sys.time()
final-inicio
```

```
## Time difference of 7.874498 secs
```

Ejercicio 5

Repetir el ejercicio anterior utilizando la cabeza de Gauss

```
system.time({
  n <- length(Secuencia_dni)

  # Aplicamos la fórmula de Gauss
  total_gauss <- (n * (n + 1)) / 2

  print(total_gauss)
})
```

```
## [1] 1225
```

```
##      user  system elapsed
##    0.001   0.000   0.000
```

Ejercicios del profe:

1.4 Generar un vector secuencia

```
# --- 1. Generar secuencia con bucle 'for' ---
# Primero inicializamos el vector
system.time({
  A <- 0
  for (i in 1:50000) {
    A[i] <- (i * 2)
  }
})
```

```
##      user  system elapsed
##    0.012   0.001   0.013
```

```
# Verificamos los resultados
head(A)
```

```
## [1]  2  4  6  8 10 12
```

```
tail(A)
```

```
## [1] 99990 99992 99994 99996 99998 100000
```

```
system.time({
  B <- seq(from = 2, to = 100000, by = 2)
})
```

```
##      user  system elapsed
##    0.001   0.000   0.001
```

```
# Verificamos los resultados
head(B)
```

```
## [1]  2  4  6  8 10 12
```

```
tail(B)
```

```
## [1] 99990 99992 99994 99996 99998 100000
```

1.5 Implementación de una serie Fibonacci o Fibonacci

```
# Definimos la cantidad de elementos de la serie que queremos calcular
n <- 30
```

```
# --- Método 1: Bucle 'for' con pre-asignación
system.time({
  fib <- numeric(n)
  fib[1] <- 0
  fib[2] <- 1

  for (i in 3:n) {
    fib[i] <- fib[i-1] + fib[i-2]
  }
})
```

```
##      user  system elapsed
##    0.003   0.000   0.003
```

```
# Ver los primeros resultados
print("Primeros números de la serie:")
```



```
## [1] "Primeros números de la serie:"
head(fib, 10)

## [1] 0 1 1 2 3 5 8 13 21 34
# --- Método 2: Función Recursiva

fib_recursiva <- function(k) {
  if (k == 0) return(0)
  if (k == 1) return(1)
  return(fib_recursiva(k - 1) + fib_recursiva(k - 2))
}

print("Tiempo ejecución recursiva:")

## [1] "Tiempo ejecución recursiva:"
system.time({
  # Calculamos el elemento n de forma individual
  resultado_rec <- fib_recursiva(n)
})

##      user  system elapsed
## 0.789    0.002    0.795
```

1.6 De nición matemática recurrente

```
# --- Resolver consigna con bucle 'while' ---

# Inicializamos la serie según la definición matemática
fib <- c(0, 1)
i <- 2 # Empezamos el conteo de iteraciones/posiciones

system.time({
  # El bucle sigue mientras el último número generado sea <= 1.000.000
  while (fib[i] <= 1000000) {
    i <- i + 1
    # Definición recurrente:  $f(n+1) = f(n) + f(n-1)$ 
    fib[i] <- fib[i-1] + fib[i-2]
  }
})

##      user  system elapsed
## 0.004    0.000    0.004

# --- Resultados ---
cat("El primer número mayor a 1.000.000 es:", fib[i], "\n")

## El primer número mayor a 1.000.000 es: 1346269
cat("Se necesitaron", i - 1, "iteraciones (pasos) para superarlo.\n")

## Se necesitaron 31 iteraciones (pasos) para superarlo.
# Mostramos los últimos elementos para verificar
tail(fib)
```

```
## [1] 121393 196418 317811 514229 832040 1346269
```

1.7 Ordenacion de un vector por metodo burbuja

```
# --- 1. Creamos un vector desordenado ---  
# Usamos un vector de 1000 números aleatorios  
set.seed(123) # Para que el resultado sea siempre el mismo  
vector_desordenado <- sample(1:1000, 1000)
```

```
# --- 2. Implementación del Método Burbuja ---  
bubble_sort <- function(x) {  
  n <- length(x)  
  # El bucle externo recorre todo el vector  
  for (i in 1:(n - 1)) {  
    # El bucle interno compara pares adyacentes  
    for (j in 1:(n - i)) {  
      if (x[j] > x[j + 1]) {  
        # Intercambio de posición (el "burbujeo")  
        temp <- x[j]  
        x[j] <- x[j + 1]  
        x[j + 1] <- temp  
      }  
    }  
  }  
  return(x)  
}
```

```
# --- 3. Medimos Performance del Burbuja vs R Nativo ---
```

```
cat("Tiempo con Método Burbuja:\n")
```

```
## Tiempo con Método Burbuja:
```

```
system.time({  
  vector_ordenado_burbuja <- bubble_sort(vector_desordenado)  
})
```

```
##      user  system elapsed  
## 0.059    0.000    0.063
```

```
cat("\nTiempo con función sort() nativa de R:\n")
```

```
##
```

```
## Tiempo con función sort() nativa de R:
```

```
system.time({  
  vector_ordenado_R <- sort(vector_desordenado)  
})
```

```
##      user  system elapsed  
## 0.000    0.000    0.001
```

1.8 La penitencia de Newton

```
# Definimos el límite de la suma (1*10^7 para que no tarde demasiado en R)
# Nota: 1*10^10 es un número extremadamente grande para un bucle for en R
n <- 1e7
```

```
# --- MÉTODO 1: La "Penitencia" (Bucle For / Fuerza Bruta) ---
# Este método suma uno por uno, como quería el profesor.
```

```
system.time({
  suma_iterativa <- 0
  for (i in 1:n) {
    suma_iterativa <- suma_iterativa + i
  }
})
```

```
##      user  system elapsed
##    0.157    0.000    0.158
```

```
cat("Resultado Suma Iterativa:", suma_iterativa, "\n")
```

```
## Resultado Suma Iterativa: 5e+13
```

```
# --- MÉTODO 2: La "Estrategia de Newton/Gauss" (Fórmula Analítica) ---
# Usamos la suma de una progresión aritmética: (n * (n + 1)) / 2
```

```
system.time({
  suma_analitica <- (n * (n + 1)) / 2
})
```

```
##      user  system elapsed
##         0         0         0
```

```
cat("Resultado Suma Analítica:", suma_analitica, "\n")
```

```
## Resultado Suma Analítica: 5e+13
```

2 Agrupación K-mean

Medición de Performance: Agrupamiento K-Means En el análisis de datos industriales, no basta con agrupar los componentes; es imperativo evaluar qué tan “buenas” son esas agrupaciones. A diferencia del aprendizaje supervisado, donde comparamos predicciones contra etiquetas reales, en el Clustering medimos la calidad mediante la cohesión interna y la separación externa.

1. Métricas de Evaluación Interna Para determinar si la segmentación de maquinaria tiene sentido estadístico, se utilizan principalmente dos indicadores: WSS (Within-Cluster Sum of Squares): Mide la suma de los cuadrados dentro de cada grupo. Representa la variabilidad interna. Un WSS bajo indica que los componentes dentro de un cluster son muy similares entre sí (alta cohesión). Coeficiente de Silueta (s_i): Es una medida que combina cohesión y separación. Su valor oscila entre -1 y 1 : 1 : El componente está muy bien ubicado en su cluster. 0 : El componente está en el límite entre dos clusters. -1 : El componente probablemente ha sido asignado al grupo incorrecto.
2. El Gráfico de Violín como Herramienta de Validación Si bien métricas como el “Método del Codo” nos ayudan a elegir el número óptimo de clusters (k), el gráfico de violín es esencial para la interpretación física de los resultados. Este gráfico combina un diagrama de caja (boxplot) con una curva de densidad de probabilidad. En el contexto de mantenimiento industrial, nos permite visualizar: Distribución y Densidad: Podemos ver dónde se concentra la mayoría de los datos de sensores (vibración, temperatura) para cada grupo. Separación de Grupos: Si los “cuerpos” de los violines de diferentes clusters no se solapan significativamente, el algoritmo ha logrado identificar estados operativos claramente distintos (ej. Estado Normal vs. Falla Inminente). Detección de Anomalías: La extensión de las “colas” del violín permite identificar máquinas que, aunque pertenecen a un grupo, presentan valores extremos que requieren atención.

```

# Librerías necesarias
library(ggplot2)
library(cluster)

# 1. Simulación de datos de sensores (Normalizados)
set.seed(123)
vibracion <- c(rnorm(100, 2, 0.5), rnorm(100, 5, 0.8), rnorm(100, 8, 0.6))
temperatura <- c(rnorm(100, 30, 5), rnorm(100, 50, 7), rnorm(100, 70, 5))
df <- data.frame(vibracion, temperatura)

# Escalamiento (Paso crítico mencionado en tu texto)
df_scaled <- scale(df)

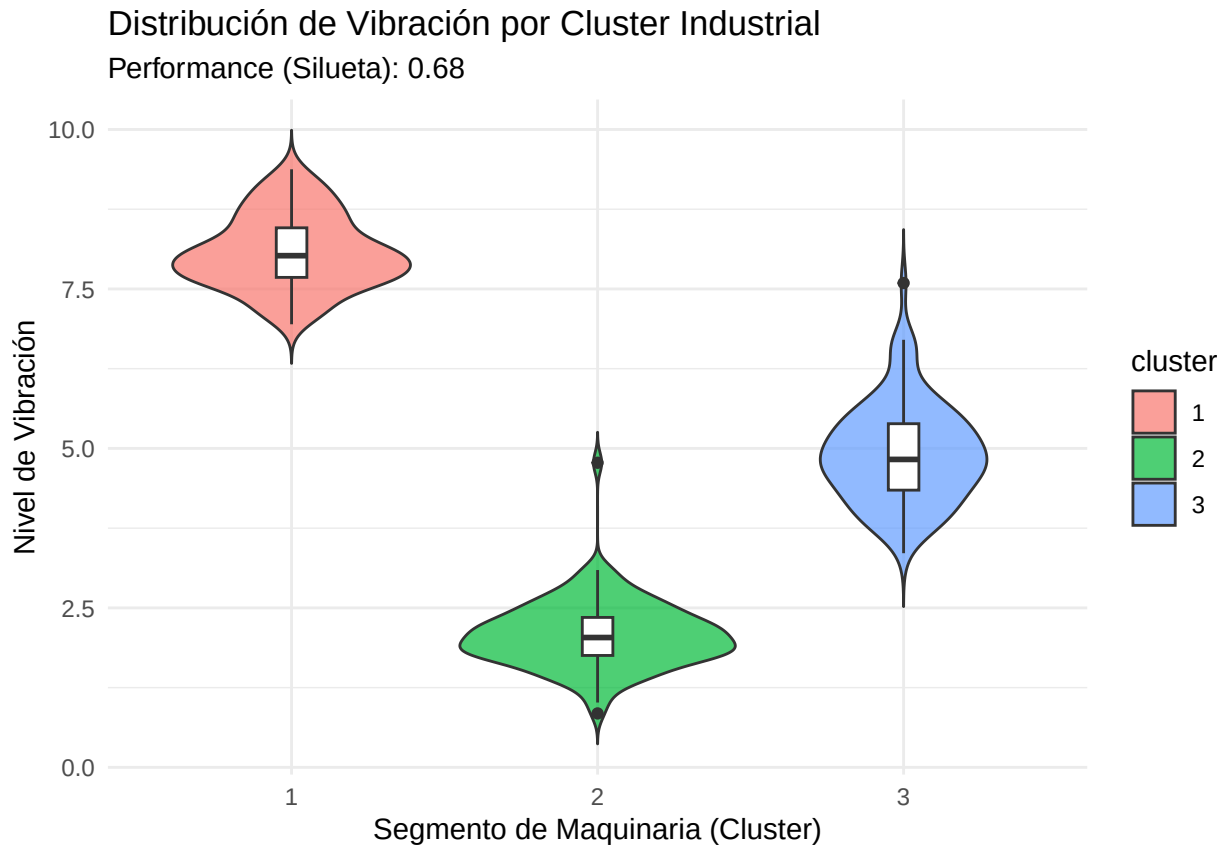
# 2. Ejecución de K-means (k=3)
fit <- kmeans(df_scaled, centers = 3, nstart = 25)
df$cluster <- as.factor(fit$cluster)

# 3. Medición de Performance: Coeficiente de Silueta
sil <- silhouette(fit$cluster, dist(df_scaled))
avg_sil <- mean(sil[, 3])
cat("El coeficiente de silueta promedio es:", avg_sil)

## El coeficiente de silueta promedio es: 0.6808356

# 4. Visualización con Gráfico de Violín
ggplot(df, aes(x = cluster, y = vibracion, fill = cluster)) +
  geom_violin(trim = FALSE, alpha = 0.7) +
  geom_boxplot(width = 0.1, fill = "white") +
  labs(title = "Distribución de Vibración por Cluster Industrial",
       subtitle = paste("Performance (Silueta):", round(avg_sil, 2)),
       x = "Segmento de Maquinaria (Cluster)",
       y = "Nivel de Vibración") +
  theme_minimal()

```



Matrices e Inversa

En este ejercicio, crearemos una matriz cuadrada de 3x3 utilizando el valor de tu DNI (506) y otros valores para asegurar que la matriz sea invertible (que su determinante sea distinto de cero). Luego, utilizaremos la función `solve()` que es la estándar en R para obtener la matriz inversa.

```
# 1. Definimos los datos de la matriz basándonos en tu DNI
# Usamos una secuencia pequeña para completar los otros campos
dni_val <- 049
datos <- c(dni_val, 10, 15,
           2, 5, 8,
           7, 1, 9)
```

```
# 2. Creamos la matriz de 3x3
A <- matrix(datos, nrow = 3, ncol = 3, byrow = TRUE)
```

```
cat("### Matriz Original (A):\n")
```

```
## ### Matriz Original (A):
```

```
print(A)
```

```
##      [,1] [,2] [,3]
## [1,]  49  10  15
## [2,]   2   5   8
## [3,]   7   1   9
```

```
# 3. Verificamos el determinante
# Una matriz tiene inversa solo si su determinante NO es 0
```

```

determinante <- det(A)
cat("\nDeterminante de la matriz:", determinante, "\n")

##
## Determinante de la matriz: 1698
# 4. Calculamos la Matriz Inversa
# En R, la función solve(A) devuelve la inversa de A
if (determinante != 0) {
  A_inversa <- solve(A)
  cat("\n### Matriz Inversa (A^-1):\n")
  print(A_inversa)

  # 5. Verificación
  # Multiplicar una matriz por su inversa debe dar la Matriz Identidad (1s en la diagonal)
  # Usamos el operador %% para multiplicación de matrices
  identidad <- round(A %% A_inversa, 2)
  cat("\n### Verificación (A * A^-1 = Identidad):\n")
  print(identidad)
} else {
  cat("\nLa matriz es singular y no tiene inversa.")
}

##
## ### Matriz Inversa (A^-1):
##           [,1]      [,2]      [,3]
## [1,]  0.02179034 -0.04416961  0.002944641
## [2,]  0.02237927  0.19787986 -0.213191991
## [3,] -0.01943463  0.01236749  0.132508834
##
## ### Verificación (A * A^-1 = Identidad):
##           [,1] [,2] [,3]
## [1,]      1   0   0
## [2,]      0   1   0
## [3,]      0   0   1

```

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

Including Plots

Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.